NoSQL Explained

4 families, BASE vs ACID, and when not to use it

Watch the 4-min video

In one line: NoSQL databases trade joins, rich transactions and ad hoc queries for scale-out and very fast known access patterns; pick one when you have one simple access pattern at huge volume, and stay relational when data is relational or the scale fits one Postgres primary.

1 Why NoSQL exists

Your app stores user sessions in Postgres. **100,000 reads and writes per second**, each a simple lookup by session ID. No joins, no reports. The database is struggling, and you pay for features you never use. That is the kind of workload NoSQL was built for.

NoSQL databases appeared in the **late 2000s**, from companies like Amazon and Google, for three reasons:

- **Scale.** Data and traffic bigger than one machine, so the data is **partitioned across many nodes** from day one.
- **Flexible schemas.** Records whose shape varies between items, or changes often.
- **Specific access patterns, served extremely fast.** The database is built around a few known queries, not every possible one.

2 The four families

"NoSQL" is not one thing. There are four main families, and each is shaped for a different kind of data. The next lessons cover each one.

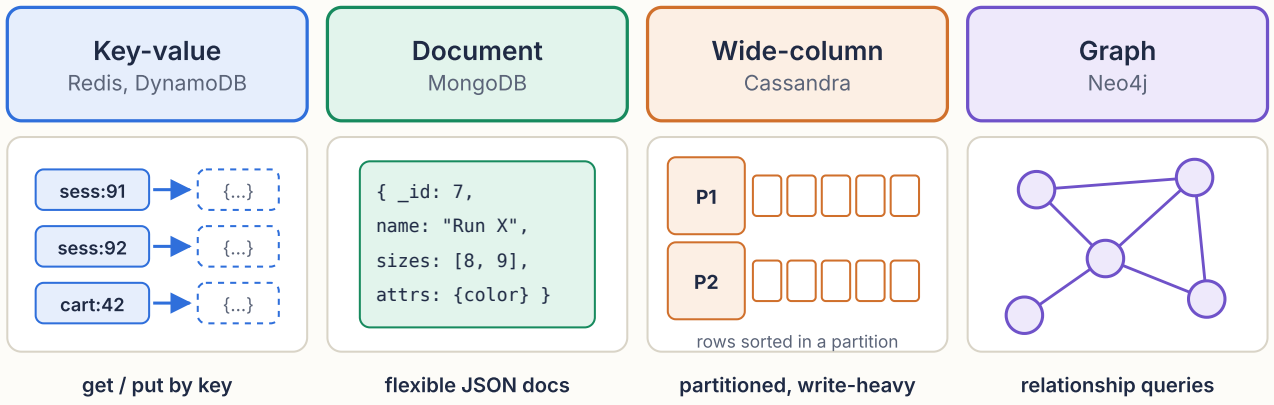


Figure 1. Key-value: a value under a key. Document: a self-contained JSON document. Wide-column: big partitions of sorted rows. Graph: nodes and the relationships between them.

3 What you give up

- **Joins, usually.** Data is **denormalized**: copied into the shape each query needs, so one read answers it.

- **Multi-row transactions, often.** They are missing or limited (for example to one partition, or slower and capped in size).
- **Ad hoc querying.** You must know your access patterns **up front** and design your keys around them. A new question can mean a new table or index.
- **Strong consistency, sometimes.** Replicas may briefly disagree.

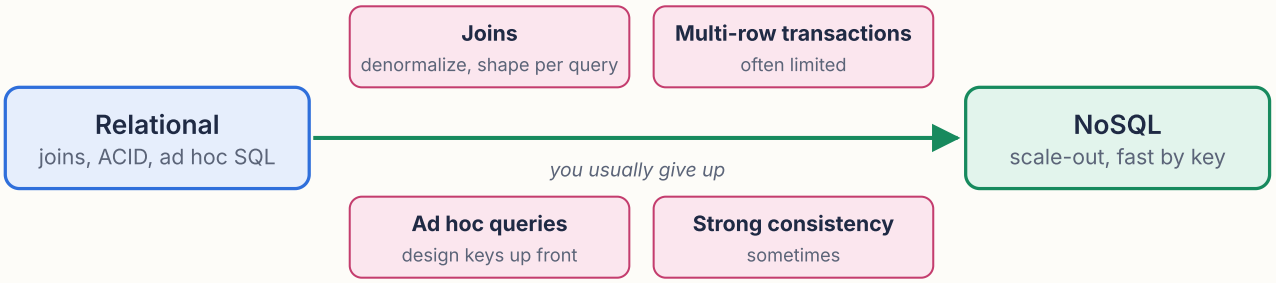


Figure 2. The trade: scale-out and speed for known queries, paid for with features the relational model gives you for free.

4 BASE vs ACID

Relational databases promise **ACID** transactions. Many NoSQL systems follow the **BASE** philosophy instead:

- **Basically available:** the system keeps answering, even during failures (some answers may be stale).
- **Soft state:** data may change over time without new input, as replicas converge.
- **Eventually consistent:** if writes stop, all replicas end up agreeing.

BASE favours **availability over immediate consistency**. But many modern NoSQL systems also offer **tunable or strong consistency**: DynamoDB has strongly consistent reads as an option, and Cassandra lets you choose a consistency level per query. It is a spectrum, not a switch.

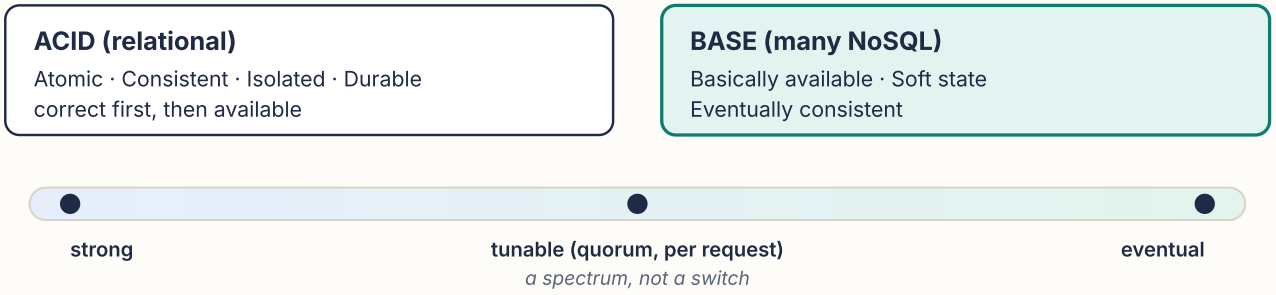


Figure 3. ACID and BASE are two ends of a range. Most real systems let you pick a point on it, sometimes per request.

5 Example: sessions in DynamoDB

Sessions are the textbook fit. The **partition key is the session ID**. The access pattern is: get by ID, put, and expire after 30 minutes, using a **time to live (TTL)** attribute. No joins are needed. It scales horizontally with no servers to manage, at single-digit milliseconds. Redis would work too, with in-memory speed.

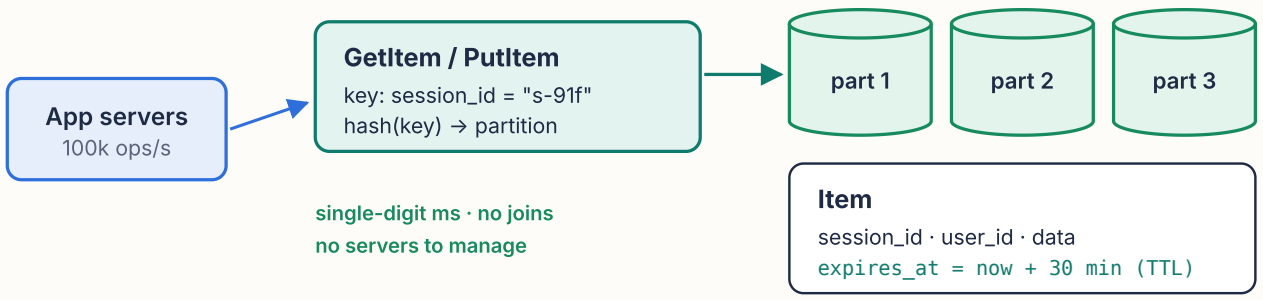


Figure 4. Each request hashes the session ID to one partition. The TTL attribute lets the database delete expired sessions for you.

WORKED EXAMPLE: WHY IT FITS

- **One access pattern:** every call is "get or put session s-91f". No query ever needs another attribute.
- **A simple key:** session IDs are random, so traffic spreads evenly across partitions; no hot spots.
- **Huge volume:** 100,000 ops/s spread evenly over, say, 200 partitions is 500 ops/s each, well under one DynamoDB partition's limit (about 3,000 reads or 1,000 writes per second). DynamoDB adds partitions as traffic grows.
- **Expiry for free:** write `expires_at = now + 30 min` on each put. No cleanup job. (DynamoDB deletes expired items in the background, not instantly, so also check `expires_at` on read.)

6 When not to use NoSQL

- **The data is relational:** many entities, joined in many ways.
- **You need multi-row transactions and strong integrity:** payments, ledgers, inventory with constraints.
- **Access patterns are unknown,** or you need flexible reporting.
- **Your scale fits on one Postgres primary with replicas.** That covers most applications.

COMMON MISTAKES

- Picking NoSQL for "flexibility" when the data is really relational, then rebuilding joins in application code.
- Designing tables before listing the queries; in NoSQL the keys come from the access patterns.
- Assuming NoSQL means "no consistency": many systems offer strong reads when you ask for them.
- Moving to NoSQL for scale you do not have yet.

7 Key terms

NoSQL: non-relational databases built to scale out

Access pattern: a query the app will run, known up front

Eventual consistency: replicas agree once writes stop

Partition key: decides which node holds a record

ACID: atomic, consistent, isolated, durable

Tunable consistency: choose strength per request

Denormalize: copy data into the shape each query needs

BASE: basically available, soft state, eventually consistent

TTL: time to live: automatic expiry of a record

8 Interview questions

Q1. What is BASE, and how does it differ from ACID? BASIC

Basically available, soft state, eventually consistent. It favours staying available during failures over immediate consistency, so replicas may briefly disagree. ACID makes every transaction atomic and isolated and every read correct, even if that costs availability.

Q2. Name the four NoSQL families and one use for each. BASIC

Key-value (Redis, DynamoDB): sessions and caches. Document (MongoDB): product catalogs. Wide-column (Cassandra): chat messages and event logs. Graph (Neo4j): recommendations and fraud rings.

Q3. What do you usually give up with NoSQL? BASIC

Joins (you denormalize), multi-row transactions (limited), ad hoc queries (keys are designed for known access patterns), and sometimes strong consistency.

Q4. Sessions in Postgres: 100,000 simple lookups per second, no joins. Move them? SCENARIO

Yes, it is a good fit: one access pattern, a random key, huge volume and natural expiry. Use DynamoDB with session_id as the partition key and a 30-minute TTL, or Redis for in-memory speed. Keep users and orders in Postgres.

Q5. A team wants MongoDB for a payments ledger "for flexibility". Your advice? SCENARIO

Keep it relational. A ledger needs multi-row transactions (debit and credit together), constraints and audit reports with ad hoc queries. Postgres does all of that, and one primary with replicas handles thousands of payments per second.

Q6. Your product manager keeps asking new reporting questions. Does that push you to NoSQL? SCENARIO

No, the other way. Unknown, changing queries suit SQL. NoSQL tables are shaped for known queries; each new question can need a new table or index. If reports are heavy, copy data to a warehouse.

Q7. Is NoSQL always eventually consistent? TRICKY

No. DynamoDB offers strongly consistent reads, Cassandra lets you choose QUORUM per query, and MongoDB defaults to reading from the primary. Consistency is a spectrum you tune.

Q8. At what point does a relational database stop being enough? TRICKY

Later than most people think. One well-tuned Postgres primary with read replicas handles tens of thousands of queries per second and terabytes of data. Move a workload to NoSQL when one simple access pattern outgrows that, not before.

Next: SD-E-014 · Key-Value Stores Explained backendroadmap.com